多线程 - 深度解析讲义

StepFun 教学组

2026 年 03 月 02 日

1 多线程

1. 以下关于线程的说法错误的是

A. 多线程可以提高程序的执行效率

B. 线程之间共享内存

C. 实现线程的方式有继承 Thread 类和实现 Runnable 接口

D. 线程是进程的一部分

答案与深度解析 答案：**B**

【核心认知框架】

题目本质：考查线程与进程的关系、线程内存模型

关键区分点：线程共享进程的堆内存，但有独立的栈空间

命题人意图：测试对线程内存模型的深入理解

【选项原理深度拆解】

A. 多线程可以提高程序的执行效率—正确（并发优势）

多核利用：在多核 CPU 上，多线程可真正并行执行

IO 等待优化：单线程等待 IO 时，其他线程可继续工作

注意：单核 CPU 下，多线程通过时间片轮转提高响应性，但不提高执行效率

B. 线程之间共享内存—错误（部分共享）

线程内存模型：

堆：所有线程共享

方法区：所有线程共享

栈：每个线程独立

程序计数器：每个线程独立

正确说法：线程共享进程的堆内存和方法区，但每个线程有独立的栈空间

代码验证：

```java
public class ThreadMemory {
    int sharedVar = 0; // 堆内存，线程共享

    public void method() {
        int localVar = 1; // 栈内存，线程私有
    }
}
```

C. 实现线程的方式有继承 **Thread** 类和实现 **Runnable** 接口—正确

方式一：继承 **Thread** 类

```java
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
new MyThread().start();
```

方式二：实现 **Runnable** 接口

```java
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable running");
    }
}
new Thread(new MyRunnable()).start();
```

方式三：实现 **Callable** 接口（带返回值）

```java
class MyCallable implements Callable<String> {
    public String call() { return "result"; }
}
FutureTask<String> task = new FutureTask<>(new MyCallable());
new Thread(task).start();
```

D. 线程是进程的一部分—正确

进程：正在执行的程序，是资源分配的基本单位

线程：CPU 调度的基本单位，是进程内的执行流

关系：一个进程可以包含多个线程，线程共享进程资源

【应背诵知识点（命题人思维版）】

进程 **vs** 线程：

对比	进程	线程
资源分配	进程是资源分配单位	线程共享进程资源
调度	进程是调度单位	线程是 CPU 调度单位
通信	进程间通信复杂	线程间通信简单

高频陷阱清单：

陷阱 1：”线程完全共享内存” → 忽略栈的独立性

【命题趋势与实战策略】

考场秒杀技巧：线程共享堆和方法区，私有栈和程序计数器

2. 在 **Java** 中，以下哪个方法用于启动线程？

A. start()

B. run()

C. sleep()

D. yield()

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：区分线程的启动方法与执行方法

关键区分点：start() 创建新线程，run() 只是普通方法

命题人意图：测试对线程生命周期的理解

【选项原理深度拆解】

A. start() —正确（启动新线程）

作用：启动新线程，JVM 调用线程的 run() 方法

```
Thread t = new Thread(() -> System.out.println("Running"));
t.start(); // 启动新线程执行run()
// 同时输出"Running"
```

线程状态变化：New → Runnable → Running → Terminated

B. run() —错误（不启动新线程）

作用：run() 只是普通方法，在当前线程执行

```
Thread t = new Thread(() -> System.out.println("Running"));
t.run(); // 在主线程执行，不会创建新线程
// 不会并发执行
```

C. sleep() —错误（暂停线程）

作用：让当前线程暂停指定时间

```
Thread.sleep(1000); // 暂停1秒
```

注意：sleep() 不释放锁，静态方法作用于当前线程

D. yield() —错误（让步）

作用：提示调度器让出 CPU 时间片

```
Thread.yield(); // 暂停当前线程，允许其他线程执行
```

注意：yield() 不确定一定让出，只是建议

【应背诵知识点（命题人思维版）】

start() vs run()：

start()：创建新线程，异步执行

run()：同步执行，不创建新线程

高频陷阱清单：

陷阱 1：调用 run() 不会启动新线程

陷阱 2：start() 只能调用一次，多次调用抛 IllegalThreadStateException

【命题趋势与实战策略】

考场秒杀技巧：记启动用 start()，执行用 run()

3. 以下关于同步的说法错误的是

A. 可以使用 synchronized 关键字实现同步

B. 同步可以保证线程安全

C. 同步会降低程序的执行效率

D. 同步可以在方法和代码块上使用

答案与深度解析 答案：**D**（原题可能存在争议）

【核心认知框架】

题目本质：考查 synchronized 同步机制的理解

关键区分点：D 选项实际上是正确的

命题人意图：本题可能有误，出题人可能想设置其他错误选项

【选项原理深度拆解】

A. 可以使用 **synchronized** 关键字实现同步—正确

synchronized：Java 内置的同步关键字

作用：保证同一时刻只有一个线程执行同步代码

B. 同步可以保证线程安全—正确

线程安全：多个线程访问共享数据时，程序行为正确

synchronized 保证：原子性、可见性、有序性

C. 同步会降低程序的执行效率—正确

原因：同步导致线程等待，增加上下文切换

优化方向：减小同步粒度、使用并发包

D. 同步可以在方法和代码块上使用—正确

方法级别：

```
public synchronized void method() { } // 锁this对象
public static synchronized void staticMethod() { } // 锁类对象
```

代码块级别：

```
synchronized(obj) { } // 锁指定对象
synchronized(this) { } // 锁当前实例
```

```
synchronized(ClassName.class) { } // 锁Class对象
```

【应背诵知识点（命题人思维版）】

同步的代价：

线程等待时间增加

上下文切换开销

可能导致死锁

高频陷阱清单：

注意：本题 D 选项实际上是正确的

【命题趋势与实战策略】

说明：本题原答案 D，但 D 选项本身正确，可能是题目有误

4. 以下哪个方法用于获取当前线程的名称？

A. getName()

B. getThreadName()

C. currentThreadName()

D. threadName()

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：考查 Thread 类的 API 方法

关键区分点：获取当前线程需要组合调用

命题人意图：测试对 Thread API 的熟悉程度

【选项原理深度拆解】

A. getName() —正确（获取线程名）

使用方式：

```
Thread.currentThread().getName(); // 获取当前线程名称
new Thread(() -> {}).getName(); // 获取指定线程名称
```

默认命名：Thread-0, Thread-1, ...

B. getThreadName() —错误（不存在）

辨析：没有此方法，只有 getName()

C. currentThreadName() —错误（不存在）

正确写法：Thread.currentThread().getName()

currentThread()：静态方法，返回当前执行的 Thread 对象

D. threadName() —错误（不存在）

辨析：常见干扰项

【应背诵知识点（命题人思维版）】

Thread 常用 API：

currentThread()：返回当前线程

getName()：获取线程名

setName(String name)：设置线程名

getId()：获取线程 ID

getPriority()：获取优先级

【命题趋势与实战策略】

考场秒杀技巧：Thread.currentThread().getName() 是标准写法

5. 以下哪个方法用于暂停当前线程指定的时间？

A. sleep()

B. wait()

C. yield()

D. stop()

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：区分线程控制方法的不同用途

关键区分点：sleep 是线程暂停，wait 是等待 notify

命题人意图：测试对线程状态转换的理解

【选项原理深度拆解】

A. sleep() —正确（线程休眠）

作用：让当前线程暂停指定时间

```
Thread.sleep(1000); // 暂停1毫秒
Thread.sleep(1000L); // 支持long类型
```

特点：

静态方法，作用于当前线程

不释放锁

会抛出 InterruptedException

B. wait() —错误（需配合锁）

作用：让线程进入等待状态

```
synchronized(obj) {
```

```
    obj.wait(); // 释放锁，等待notify
}
```

特点：必须在 synchronized 块中调用

C. yield() —错误（只是建议）

作用：提示调度器让出 CPU（不一定成功）

D. stop() —错误（已废弃）

说明：stop() 已废弃，会导致数据不一致

替代方案：使用 interrupt() + 标志位

【应背诵知识点（命题人思维版）】

(a) **sleep vs wait**：

对比	**sleep**	**wait**
所属类	Thread	Object
释放锁	否	是
用法限制	无	必须在 synchronized 中

【命题趋势与实战策略】

考场秒杀技巧：sleep 暂停指定时间，wait 等待 notify

6. 以下关于 **Java** 中线程同步的方法错误的是

A. 使用 synchronized 关键字修饰方法

B. 使用 synchronized 关键字修饰代码块

C. 使用 ReentrantLock 类

D. 多线程访问共享资源不需要同步

答案与深度解析 答案：**D**

【核心认知框架】

题目本质：考查多线程同步的必要性

关键区分点：共享资源只要有写操作就必须同步

命题人意图：测试对线程安全条件的理解

【选项原理深度拆解】

A. 使用 **synchronized** 关键字修饰方法—正确

实例方法：锁 this 对象

静态方法：锁 Class 对象

B. 使用 **synchronized** 关键字修饰代码块—正确

更细粒度：可以只同步关键代码

C. 使用 **ReentrantLock** 类—正确

显式锁：JDK5+ 提供的显式锁

```java
Lock lock = new ReentrantLock();
lock.lock();
try {
    // 临界区
} finally {
    lock.unlock();
}
```

D. 多线程访问共享资源不需要同步—错误

共享资源：多个线程同时访问的变量/对象

必须同步的情况：

至少有一个线程对共享资源进行写操作

存在竞态条件

不需要同步：只读操作或使用线程安全类

【应背诵知识点（命题人思维版）】

线程安全条件：

多个线程共享数据

至少有一个线程修改数据

高频陷阱清单：

陷阱 1：”只读不需要同步” → 正确

陷阱 2：”有写操作就一定要同步” → 正确

【命题趋势与实战策略】

考场秒杀技巧：共享资源有写必须同步，只读无需同步

7. 以下关于 **Java** 中线程通信的说法错误的是

A. 可以使用 wait()、notify() 和 notifyAll() 方法

B. 这些方法必须在 synchronized 代码块中使用

C. wait() 会释放锁

D. notify() 会唤醒所有等待的线程

答案与深度解析 答案：**D**

【核心认知框架】

题目本质：考查 Object 的 wait/notify 机制

关键区分点：notify 只唤醒一个，notifyAll 唤醒全部

命题人意图：测试对线程通信机制的深入理解

【选项原理深度拆解】

A. 可以使用 **wait()**、**notify()** 和 **notifyAll()** 方法—正确

Object 方法：定义在 java.lang.Object 中

B. 这些方法必须在 **synchronized** 代码块中使用—正确

原因：调用 wait/notify 需要持有对象的监视器锁

IllegalMonitorStateException：未持有锁时调用会抛异常

C. wait() 会释放锁—正确

与 **sleep** 的区别：wait() 释放锁，sleep() 不释放

D. notify() 会唤醒所有等待的线程—错误

notify()：只唤醒一个等待线程（由 JVM 选择）

notifyAll()：唤醒所有等待线程

```
synchronized(obj) {
    obj.notify(); // 唤醒一个线程
    // 或
    obj.notifyAll(); // 唤醒所有线程
}
```

【应背诵知识点（命题人思维版）】

(a) **wait/notify** 速查表：

方法	作用	备注
wait()	等待，释放锁	需在 synchronized 中
notify()	唤醒一个线程	不释放锁
notifyAll()	唤醒所有线程	不释放锁

【命题趋势与实战策略】

考场秒杀技巧：notify 一个，notifyAll 全部

8. 以下关于 **Java** 中线程池的说法正确的是

A. 可以减少创建和销毁线程的开销

B. 性能比手动创建线程差

C. 不能控制线程数量

D. 不需要管理线程资源

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：考查线程池的核心优势

关键区分点：线程池复用线程，减少开销

命题人意图：测试对并发编程优化的理解

【选项原理深度拆解】

A. 可以减少创建和销毁线程的开销—正确（核心优势）
线程创建开销：
分配栈内存
注册线程到 JVM
调用操作系统 API 创建线程
线程池优势：复用已有线程，避免重复创建销毁
B. 性能比手动创建线程差—错误
事实：线程池性能更好，尤其在大量短任务场景
C. 不能控制线程数量—错误
线程池参数：
corePoolSize：核心线程数
maximumPoolSize：最大线程数
D. 不需要管理线程资源—错误
恰恰相反：线程池就是为了管理线程资源
【应背诵知识点（命题人思维版）】
线程池核心优势：
复用线程，减少开销
控制并发数量
提供任务队列管理
【命题趋势与实战策略】
考场秒杀技巧：线程池 = 复用 + 控制 + 管理

9. 以下哪个是创建固定大小线程池的方法？

A. newCachedThreadPool()

B. newFixedThreadPool()

C. newScheduledThreadPool()

D. newSingleThreadExecutor()

答案与深度解析 答案：**B**
【核心认知框架】
题目本质：考查 Executors 工厂类的线程池创建方法
关键区分点：不同方法创建不同类型的线程池
命题人意图：测试对线程池类型的掌握
【选项原理深度拆解】
A. newCachedThreadPool() —可缓存线程池

特点：

核心线程数为 0

最大线程数 Integer.MAX_VALUE

线程空闲 60 秒后回收

B. newFixedThreadPool() —固定大小线程池

特点：

固定线程数

超出核心线程数的任务进入队列等待

```
ExecutorService pool = Executors.newFixedThreadPool(5);
```

C. newScheduledThreadPool() —定时任务线程池

特点：支持定时和周期性任务

D. newSingleThreadExecutor() —单线程池

特点：只有一个工作线程，保证顺序执行

【应背诵知识点（命题人思维版）】

(a) **Executors** 工厂方法：

方法	类型	适用场景
newFixedThreadPool(n)	固定大小	任务并发量稳定
newCachedThreadPool()	可扩展	任务量变化大
newScheduledThreadPool()	定时任务	定时执行任务
newSingleThreadExecutor()	单线程	任务需顺序执行

【命题趋势与实战策略】

考场秒杀技巧：固定大小 = newFixedThreadPool(n)

10. 以下关于 **Java** 中锁的说法错误的是

A. ReentrantLock 是可重入锁

B. 锁可以提高并发性能

C. 读多写少的场景适合使用读写锁

D. 锁可能导致死锁

答案与深度解析 答案：**B**

【核心认知框架】

题目本质：考查 Java 锁的高级特性

关键区分点：不当使用锁会降低并发性能

命题人意图：测试对锁的理解深度

【选项原理深度拆解】

A. ReentrantLock 是可重入锁—正确

可重入：同一线程可多次获取同一把锁

```
synchronized void method1() {
    method2(); // 可以进入，因为是同一线程
}
synchronized void method2() {
    // 同一线程可重入
}
```

B. 锁可以提高并发性能—错误

正确理解：

过度同步会降低并发性能

锁竞争导致线程等待

应使用细粒度锁或无锁算法

C. 读多写少的场景适合使用读写锁—正确

ReentrantReadWriteLock：

读锁：共享锁，多线程同时读

写锁：独占锁，排他写

D. 锁可能导致死锁—正确

死锁条件：

互斥条件

占有并等待

不可抢占

循环等待

【应背诵知识点（命题人思维版）】

锁优化策略：

减小锁粒度

使用读写锁

使用 ConcurrentHashMap

使用原子类

高频陷阱清单：

陷阱 1：锁不是越多越好，过度同步降低性能

【命题趋势与实战策略】

考场秒杀技巧：锁会降低并发性能，不是提高

11. 以下关于 **Java** 中 **volatile** 关键字的说法错误的是

A. 保证变量的可见性

B. 不能保证原子性

C. 可以替代锁

D. 常用于多线程环境

答案与深度解析 答案：**C**

【核心认知框架】

题目本质：考查 volatile 与锁的区别

关键区分点：volatile 只能保证可见性，不能保证原子性

命题人意图：测试对 volatile 的理解深度

【选项原理深度拆解】

A. 保证变量的可见性—正确

可见性：一个线程修改后，其他线程立即可见

JMM 保证：volatile 写后立即刷新到主内存

B. 不能保证原子性—正确

原子性：不可分割的操作

volatile 局限：i++ 这类复合操作不是原子的

C. 可以替代锁—错误

替代锁的条件：

对变量的操作是原子性的（单次读写）

不依赖变量的当前值

错误示例：

```
volatile int count = 0;
count++; // 不是原子操作，volatile不能保证线程安全
```

D. 常用于多线程环境—正确

适用场景：

状态标记

双重检查锁定（单例模式）

取消标志

【应背诵知识点（命题人思维版）】

(a) **volatile vs synchronized**：

对比	**volatile**	**synchronized**
可见性	保证	保证
原子性	不保证	保证
性能	高	低

(b) 高频陷阱清单：

陷阱 1：volatile 不能替代 synchronized

陷阱 2：i++ 不是原子操作

【命题趋势与实战策略】
考场秒杀技巧：volatile 保可见，不保原子，不能替代锁

12. 以下关于 **Java** 中 **Atomic** 类的说法正确的是

A. 提供了原子操作

B. 性能比锁差

C. 不能用于并发场景

D. 只支持整数类型

答案与深度解析 答案：**A**
【核心认知框架】
题目本质：考查 java.util.concurrent.atomic 包
关键区分点：原子类提供无锁原子操作
命题人意图：测试对并发工具类的了解
【选项原理深度拆解】
A. 提供了原子操作—正确（核心优势）
常用类：
AtomicInteger、AtomicLong、AtomicBoolean
AtomicReference、AtomicStampedReference
AtomicIntegerArray、AtomicLongArray

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // 原子+1
count.getAndIncrement(); // +1后返回
count.compareAndSet(0, 1); // CAS操作
```

B. 性能比锁差—错误
CAS 优势：无锁算法，通常比 synchronized 性能好
C. 不能用于并发场景—错误
恰恰相反：专为并发场景设计
D. 只支持整数类型—错误
支持类型：整数、布尔、引用、数组等
【应背诵知识点（命题人思维版）】
Atomic 类优势：
无锁算法，性能好
支持多种类型
提供 CAS 操作

【命题趋势与实战策略】
考场秒杀技巧：Atomic = 无锁 + 原子 + 高性能

13. 以下关于 **Java** 中 **ConcurrentHashMap** 的说法错误的是

A. 线程安全的 HashMap

B. 性能比 HashTable 好

C. 不支持并发遍历

D. 可以在多线程环境中使用

答案与深度解析 答案：**C**
【核心认知框架】
题目本质：考查 ConcurrentHashMap 的特性
关键区分点：支持并发遍历，迭代器弱一致性
命题人意图：测试对并发集合类的了解
【选项原理深度拆解】
A. 线程安全的 **HashMap** —正确
实现：JDK8 使用 CAS + synchronized + 红黑树
B. 性能比 **HashTable** 好—正确
原因：
HashTable：全局锁
ConcurrentHashMap：分段锁 → 细粒度锁
C. 不支持并发遍历—错误
支持并发遍历：迭代器具有弱一致性
不抛 **ConcurrentModificationException**
D. 可以在多线程环境中使用—正确
设计目标：高并发场景
【应背诵知识点（命题人思维版）】

(a) **ConcurrentHashMap vs HashTable:**

对比	**ConcurrentHashMap**	**HashTal**
锁机制	分段锁/CAS	synchronized
并发性能	高	低
迭代器弱一致	支持	不支持

【命题趋势与实战策略】
考场秒杀技巧：并发 Map 首选 ConcurrentHashMap

14. 以下关于 **Java** 中 **BlockingQueue** 的说法正确的是

A. 是一个阻塞队列

B. 不支持多线程操作

C. 元素取出顺序和插入顺序相同

D. 容量无限

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：考查 BlockingQueue 阻塞队列特性

关键区分点：队列是否阻塞取决于实现

命题人意图：测试对并发队列的掌握

【选项原理深度拆解】

A. 是一个阻塞队列—正确

阻塞特性：

take()：队列为空时阻塞

put()：队列满时阻塞

B. 不支持多线程操作—错误

恰恰相反：专为多线程设计

C. 元素取出顺序和插入顺序相同—错误

ArrayBlockingQueue：FIFO，顺序相同

PriorityBlockingQueue：按优先级，不保证顺序

D. 容量无限—错误

可设置容量：LinkedBlockingQueue 可选有界/无界

【应背诵知识点（命题人思维版）】

(a) **BlockingQueue** 方法：

方法	队列满/空时	返回值
add()	抛异常	元素
offer()	返回 false	boolean
put()	阻塞	void
poll()	返回 null	元素
take()	阻塞	元素

【命题趋势与实战策略】

考场秒杀技巧：BlockingQueue = 阻塞 + 线程安全 + 队列

15. 以下哪个是 **BlockingQueue** 的实现类？

A. ArrayBlockingQueue

B. LinkedBlockingQueue

C. PriorityBlockingQueue

D. 以上都是

答案与深度解析 答案：**D**
【核心认知框架】
题目本质：考查 BlockingQueue 的常用实现
关键区分点：三个都是常用实现
命题人意图：测试对并发队列实现的了解
【选项原理深度拆解】
A. ArrayBlockingQueue —正确
特点：有界、FIFO、数组实现
B. LinkedBlockingQueue —正确
特点：可选有界/无界、链表实现
C. PriorityBlockingQueue —正确
特点：无界、优先级队列
D. 以上都是—正确
都是 **BlockingQueue** 实现
【应背诵知识点（命题人思维版）】
BlockingQueue 实现类：
ArrayBlockingQueue：有界、FIFO
LinkedBlockingQueue：无界/有界、FIFO
PriorityBlockingQueue：无界、优先级
SynchronousQueue：零容量、直接交付
【命题趋势与实战策略】
考场秒杀技巧：Array/Linked/Priority 都是 BlockingQueue 实现

16. 以下关于 **Java** 中 **CountDownLatch** 的说法错误的是

A. 可以实现线程等待

B. 可以设置等待的线程数量

C. 等待的线程被唤醒后不能再次等待

D. 常用于多线程协作

答案与深度解析 答案：**C**（原题可能存在争议）
【核心认知框架】
题目本质：考查 CountDownLatch 一次性特性
关键区分点：是一次性的，但" 不能再次等待" 说法有歧义
命题人意图：测试对并发工具的掌握

【选项原理深度拆解】

A. 可以实现线程等待—正确

await()：等待计数归零

B. 可以设置等待的线程数量—正确

构造方法：CountDownLatch(int count)

C. 等待的线程被唤醒后不能再次等待—有争议

一次性：计数归零后不能重置

可新建实例：需要再次等待则创建新实例

D. 常用于多线程协作—正确

典型场景：主线程等待子线程完成

【应背诵知识点（命题人思维版）】

CountDownLatch vs CyclicBarrier:

对比	**CountDownLatch**	**CyclicBarrier**
能否重置	不能	能
计数方向	减到 0	增到指定值

【命题趋势与实战策略】

说明：本题原答案 C，CountDownLatch 是一次性的

17. 以下关于 **Java** 中 **CyclicBarrier** 的说法正确的是

A. 可以循环使用

B. 只能使用一次

C. 不能设置参与的线程数量

D. 不支持线程等待

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：考查 CyclicBarrier 可循环特性

关键区分点：可重置后重复使用

命题人意图：测试对并发工具的区分

【选项原理深度拆解】

A. 可以循环使用—正确（核心特性）

重置方法：reset()

```
CyclicBarrier barrier = new CyclicBarrier(3);
barrier.await(); // 3个线程都到达后继续
barrier.reset(); // 可以重复使用
```

B. 只能使用一次—错误

与 **CountDownLatch** 区别：CyclicBarrier 可重复使用

C. 不能设置参与的线程数量—错误

构造方法：CyclicBarrier(int parties)

D. 不支持线程等待—错误

恰恰相反：await() 就是让线程等待

【应背诵知识点（命题人思维版）】

CyclicBarrier 特点：

可循环使用

设置线程数量

线程相互等待

【命题趋势与实战策略】

考场秒杀技巧：CyclicBarrier = 可循环 + 相互等待

18. 以下关于 **Java** 中 **Semaphore** 的说法错误的是

A. 用于控制资源的访问数量

B. 可以实现信号量的获取和释放

C. 不能用于实现互斥

D. 是一种并发工具类

答案与深度解析 答案：**C**

【核心认知框架】

题目本质：考查 Semaphore 信号量机制

关键区分点：可初始化为 1 实现互斥

命题人意图：测试对信号量的理解

【选项原理深度拆解】

A. 用于控制资源的访问数量—正确

构造方法：Semaphore(int permits)

B. 可以实现信号量的获取和释放—正确

acquire()：获取

release()：释放

C. 不能用于实现互斥—错误

互斥实现：初始化为 1

```
Semaphore semaphore = new Semaphore(1);
semaphore.acquire(); // 获取锁
// 临界区
```

```
semaphore.release(); // 释放锁
```

D. 是一种并发工具类—正确

位置：java.util.concurrent.Semaphore

【应背诵知识点（命题人思维版）】

Semaphore 应用场景：

限流：控制同时访问资源的线程数

互斥：permits=1 时作为互斥锁

【命题趋势与实战策略】

考场秒杀技巧：Semaphore permits=1 可作互斥锁

19. 以下关于 **Java** 中 **Future** 和 **Callable** 的说法正确的是

A. Future 用于获取异步任务的结果

B. Callable 不能返回结果

C. Future 不能取消任务

D. Callable 不能抛出异常

答案与深度解析 答案：**A**

【核心认知框架】

题目本质：考查 Future/Callable 异步编程

关键区分点：Future 获取结果，Callable 执行任务

命题人意图：测试对异步编程的理解

【选项原理深度拆解】

A. Future 用于获取异步任务的结果—正确

方法：

- get()：阻塞获取结果
- get(long timeout, TimeUnit unit)：超时获取
- isDone()：判断是否完成

B. Callable 不能返回结果—错误

返回结果：V call() throws Exception

C. Future 不能取消任务—错误

cancel()：取消任务

```
Future<?> future = executor.submit(task);
future.cancel(true); // 尝试取消
```

D. Callable 不能抛出异常—错误

可抛出异常：call() throws Exception
【应背诵知识点（命题人思维版）】
Future/Callable 关系：
Callable：类似 Runnable，但有返回值
Future：获取 Callable 的执行结果
提交：ExecutorService.submit(Callable)
【命题趋势与实战策略】
考场秒杀技巧：Callable 有返回，Future 获取结果，可取消

20. 以下关于 **Java** 中 **CompletableFuture** 的说法错误的是

A. 是 Future 的增强版

B. 支持异步回调

C. 性能比 Future 差

D. 提供了丰富的组合操作

答案与深度解析 答案：**C**
【核心认知框架】
题目本质：考查 CompletableFuture 异步编程
关键区分点：功能强大，性能不差
命题人意图：测试对现代异步编程的了解
【选项原理深度拆解】
A. 是 **Future** 的增强版—正确
继承：实现 Future + CompletionStage
B. 支持异步回调—正确
thenApply()：转换结果
thenAccept()：消费结果
thenCompose()：组合异步
C. 性能比 **Future** 差—错误
性能：CompletableFuture 不差于 Future
优势：非阻塞 + 组合
D. 提供了丰富的组合操作—正确
链式调用：thenApply、thenCompose、thenCombine 等
【应背诵知识点（命题人思维版）】
CompletableFuture 优势：
非阻塞异步编程

丰富的组合操作

支持函数式回调

创建方式：

```
CompletableFuture.supplyAsync(() -> "result")
    .thenApply(s -> s.toUpperCase())
    .thenAccept(System.out::println);
```

【命题趋势与实战策略】

考场秒杀技巧：CompletableFuture = Future + 异步回调 + 组合操作